Practical-1

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes. Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

OUTPUT :

```
v_santosh@DESKTOP-E34M7RI:~$ mkdir practical
mkdir: practical: File exists
v_santosh@DESKTOP-E34M7RI:~$ mkdir Practical_01
v_santosh@DESKTOP-E34M7RI:~$ cd Practical_01
v_santosh@DESKTOP-E34M7RI:~/Practical_01$ nano fork_exec.c
v_santosh@DESKTOP-E34M7RI:~/Practical_01$ gcc fork_exec.c -o fork_exec
v_santosh@DESKTOP-E34M7RI:~/Practical_01$ ./fork_exec
====================================
 Linux Command Execution using fork()
====================================
Enter a Linux command: ls

Parent Process
Parent PID : 711
Waiting for Child Process...

Child Process
Child PID  : 712
Parent PID : 711

Executing command...

fork_exec  fork_exec.c

Child Process Finished Successfully.
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("====================================\n");
    printf(" Linux Command Execution using fork()\n");
    printf("====================================\n");

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nExecuting command...\n\n");

        execlp(command, command, NULL);

        // Executes only if command is invalid
        perror("Execution Failed");
        exit(1);
    }
    else
    {
        // Parent Process
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());

        printf("Waiting for Child Process...\n");

        wait(NULL);

        printf("\nChild Process Finished Successfully.\n");
    }

    return 0;
}
```